Algoritmos de ordenação

Selection Sort e Merge Sort

Professor: Samuel Basílio

Aluno: João Flávio de Toledo Arruda

Selection Sort

- É a ordenação por seleção
- Transfere o menor valor do vetor para a primeira posição
- O segundo menor pra segunda posição
- E assim N-1 vezes

Vantagens

- Implementação simples
- Não necessita de vetor auxiliar, com isso, menos memoria
- Um dos mais velozes em vetores pequenos

Desvantagens

- Bem lento para vetores grandes
- Não é estável
- Faz sempre $(N^2 - N)/2$ comparações

Ilustração

i=0 8 5 2 6 9 3 1 4 0 7

Ilustração

i=1	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7

Ilustração

i=2	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7

Ilustração

i=3	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	6	9	3	5	4	8	7

Ilustração

i=4	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7

Ilustração

i=5	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7
	0	1	2	3	4	6	5	9	8	7

Ilustração

i=6	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7
	0	1	2	3	4	6	5	9	8	7
	0	1	2	3	4	5	6	9	8	7

Ilustração

i=7	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7
	0	1	2	3	4	6	5	9	8	7
	0	1	2	3	4	5	6	9	8	7

Ilustração

i=8	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7
	0	1	2	3	4	6	5	9	8	7
	0	1	2	3	4	5	6	9	8	7
	0	1	2	3	4	5	6	7	8	9

Ilustração

i=9	8	5	2	6	9	3	1	4	0	7
	0	5	2	6	9	3	1	4	8	7
	0	1	2	6	9	3	5	4	8	7
	0	1	2	3	9	6	5	4	8	7
	0	1	2	3	4	6	5	9	8	7
	0	1	2	3	4	5	6	9	8	7
	0	1	2	3	4	5	6	7	8	9
	0	1	2	3	4	5	6	7	8	9

Código

```
void selection_sort(int num[], int tam) {
  int i, j, min, aux;
  for (i = 0; i < (tam-1); i++)
  {
     min = i;
     for (j = (i+1); j < tam; j++) {
       if(num[j] < num[min])
         min = j;
     }
     if (num[i] != num[min]) {
       aux = num[i];
       num[i] = num[min];
       num[min] = aux;
     }
  }
}
```

Análise de complexidade

Podemos entender que não existe um melhor caso mesmo que o vetor esteja ordenado serão executadas todas as comparações.

$O(n^2)$ complexidade assintótica

$$c(n) = \frac{n^2}{2} - \frac{n}{2}$$

$$m(n) = 3(n-1)$$

Merge Sort

Merge Sort

- Ordenação por mistura
- Divide o problema em subproblemas e resolve-os recursivamente
- Une os pequenos problemas até finalizar a sequência

Vantagens

- Melhor caso $O(n \log n)$
- Pior caso $O(n \log n)$
- Estável

Desvantagens

- Possui recursão
- Gasto extra de memória pois é criada uma cópia do vetor para cada nível da chamada recursiva

Ilustração

Vetor original com elemento desordenados.

0	1	2	3	4	5	6	7	8
53	25	46	32	23	37	41	17	10

Ilustração

O vetor original é subdividido em dois vetores.

0	1	2	3	4	5	6	7	8
53	25	46	32	23	37	41	17	10

Ilustração

Cada um dos subvetores é novamente dividido.

0	1	2	3	4	5	6	7	8
53	25	46	32	23	37	41	17	10

Ilustração

E assim por diante.

0	1	2	3	4	5	6	7	8
53	25	46	32	23	37	41	17	10
53	25	46	32	23	37	41	17	10

Ilustração

Fusão ordenada dos subvetores

0	1	2	3	4	5	6	7	8
25	53	46	32	23	37	41	17	10

Ilustração

O subvetor (25, 53) com o subvetor (46).

0	1	2	3	4	5	6	7	8
25	46	53	32	23	37	41	17	10

Ilustração

O subvetor (32) com o subvetor (23).

0	1	2	3	4	5	6	7	8
25	46	53	23	32	37	41	17	10

Ilustração

O subvetor (25,46,53) com o subvetor (23,32).

0	1	2	3	4	5	6	7	8
23	25	32	46	53	37	41	17	10

Ilustração

O mesmo processo se repete no subvetor (37, 41, 17, 10).

0	1	2	3	4	5	6	7	8
23	25	32	46	53	37	41	17	10
23	25	32	46	53	37	41	10	17
23	25	32	46	53	10	17	37	41

Ilustração

s subvetores resultantes (23,25, 32, 36,53) e (10, 17, 37, 41) são fundidos ordenados durante a fusão.

0	1	2	3	4	5	6	7	8
10	17	23	25	32	37	41	46	53

Código

```
void mergeSort(int *V, int inicio, int fim){
    int meio;
    if(inicio < fim){
        meio = floor((inicio+fim)/2);
        mergeSort(V,inicio,meio);
        mergeSort(V,meio+1,fim);
        merge(V,inicio,meio,fim);
    }
}
```

```c
void merge(int *V, int inicio, int meio, int fim){
    int *temp, p1, p2, tamanho, i, j, k;
    int fim1 = 0, fim2 = 0;
    tamanho = fim-inicio+1;
    p1 = inicio;
    p2 = meio+1;
    temp = (int *) malloc(tamanho*sizeof(int));
    if(temp != NULL){
        for(i=0; i<tamanho; i++){
            if(!fim1 && !fim2){
                if(V[p1] < V[p2])
                    temp[i]=V[p1++];
                else
                    temp[i]=V[p2++];

                if(p1>meio) fim1=1;
                if(p2>fim) fim2=1;
            }else{
                if(!fim1)
                    temp[i]=V[p1++];
                else
                    temp[i]=V[p2++];
            }
        }
        for(j=0, k=inicio; j<tamanho; j++, k++)
            V[k]=temp[j];
    }
    free(temp);
}
```

Analise de Complexidade

- Merge é mais rápido e eficiente quando é utilizado sobre uma grande quantidade de dados

$O(n \log n)$ Complexidade linearítmica

$O(n \log n)$

Referências bibliográficas

- ZIVIANI, N. **Projeto de algoritmos** com implementações em Pascal e C. 4. ad. São Paulo:Pioneira,1999. E-Book. ISBN 85-221. Disponível em: <clip2net.com/clip/m1076/1206625061-ebook-projetos-de-algoritmos-com-implementazhes-em-pascal-e-c-nivio-ziviani-4ed-3618kb.pdf livro ziviani ebook>. Acesso em: 02/09/2019.
- *Cormen, Thomas; Leiserson, Charles; Rivest, Ronald; Stein, Clifford (2012). Algoritmos: teoria e prática 3 ed. Rio de Janeiro: Elsevier. ISBN 9788535236996*